

AI Project: Connect Four

Martin Scheuchenpflug Niklas Etzlinger

SS 2026

Experimental Setup

Game

- Connect Four on a 6×8 board (later: 6×7)
- Computer plays as second player (red)
- Fixed search depth: 4

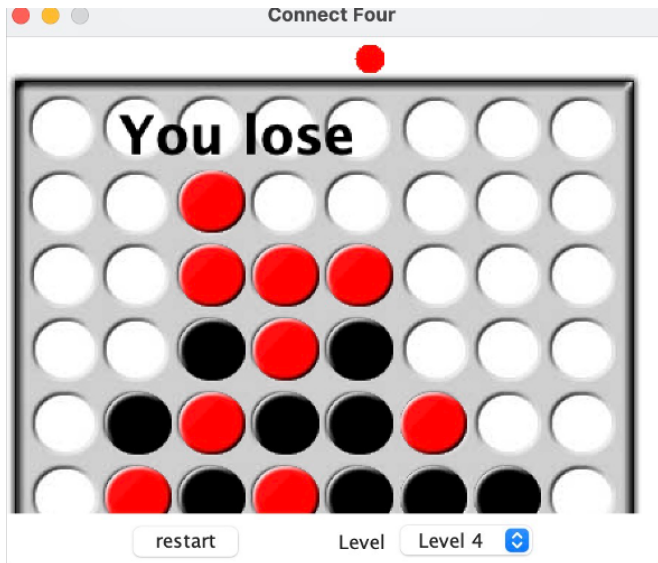
Metric

Number of board positions the computer examines before choosing its move (logged as “moves tried”).

Six variants measured

- Q1 — original (shallow α - β)
- Q2 — pure minimax (no pruning)
- Q3 — equal-value cutoffs ($\geq/\leq$)
- Q4 — deep α - β
- Q5a — pure minimax + middle-out
- Q5b — deep α - β + middle-out

The Game: Connect Four



Q1: Shallow α - β Pruning

How it works

Each recursive call passes only the *direct parent's* current best value as a bound. Bounds from grandparents and higher are silently discarded.

- Max node prunes if child strength $>$ parent min's current minimum
- Min node prunes if child strength $<$ parent max's current maximum
- Ancestors offer *no* additional tightening

Measured move counts

Move	Q1
1	1426
2	925
3	654
4	809
5	1221

Range: $\approx 650 - 2,459$ depending on board state.

Q2: Removing All Pruning

Code change

Both cutoff blocks were commented out:

```
// in expandMaxNode:
// if (strength > parentMinimum) {
//   board.undoLastMove();
//   return strength;
// }

// in expandMinNode:
// if (strength < parentMaximum) {
//   board.undoLastMove();
//   return strength;
// }
```

Effect

Every node in the search tree is expanded — no cutoffs at all.

Theoretical count (depth d , branching n)

$$\text{total}(d) = \frac{n^{d+1} - n}{n - 1}$$

For $n = 8$, $d = 4$:

$$\frac{8^5 - 8}{7} = \frac{32760}{7} = \mathbf{4680}$$

First three moves all measured exactly 4680.

Q3: Pruning on Equal Values

Original (strict)

A value *equal* to the bound does not trigger a cutoff.

```
// max node
if (strength > parentMinimum) return;
// min node
if (strength < parentMaximum) return;
```

Q3: equal cutoff

An equality does trigger a cutoff too.

```
// max node - fires on tie too
if (strength >= parentMinimum) return;
// min node - fires on tie too
if (strength <= parentMaximum) return;
```

Why it helps

The heuristic assigns identical scores to many symmetric positions early in the game. Pruning on equality eliminates entire subtrees that the strict version still explores.

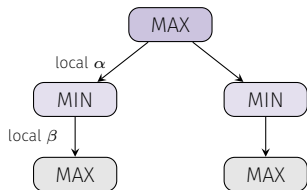
Result

Move counts drop consistently vs Q1.

Example: move 1 drops from 1426 (Q1) to 1208 (Q3).

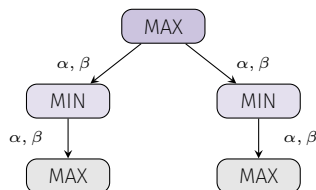
Q4: Deep α - β --- Motivation

Shallow (Q1/Q3)



Each node passes only its *own* value — ancestor bounds are lost.

Deep (Q4)



Both α and β thread through the *whole* tree — every node sees the tightest window from all ancestors.

Q4: Deep α - β --- Code

```
private int expandMaxNode(int depth, int alpha, int beta) {
    for each move:
        int strength = expandMinNode(depth - 1, alpha, beta); // pass both down
        if (strength >= beta) { board.undoLastMove(); return strength; } // beta
            cutoff
        if (strength > maxStrength) maxStrength = strength;
        if (strength > alpha) alpha = strength; // tighten alpha for next sibling
    }
private int expandMinNode(int depth, int alpha, int beta) {
    for each move:
        int strength = expandMaxNode(depth - 1, alpha, beta); // pass both down
        if (strength <= alpha) { board.undoLastMove(); return strength; } //
            alpha cutoff
        if (strength < minStrength) minStrength = strength;
        if (strength < beta) beta = strength; // tighten beta for next sibling
    }
```

Effect

Move 1: 993 (vs 1208 in Q3). Tightening at any level immediately benefits *all* descendants.

Q5: Middle-Out Move Ordering

Idea

α - β is most effective when the *best* moves come first.

In Connect Four, centre columns are generally stronger.

⇒ Try them first.

Q5a — pure minimax + middle-out

Without pruning, traversal order does not affect node count.
Counts stay close to 4680 (same formula as Q2).

Minor deviations: different move choices lead to slightly different board states.

Q5b — deep α - β + middle-out

Move 1: 302 (vs 993 in Q4).

Best columns first ⇒ tight window established immediately ⇒ remaining columns pruned early.

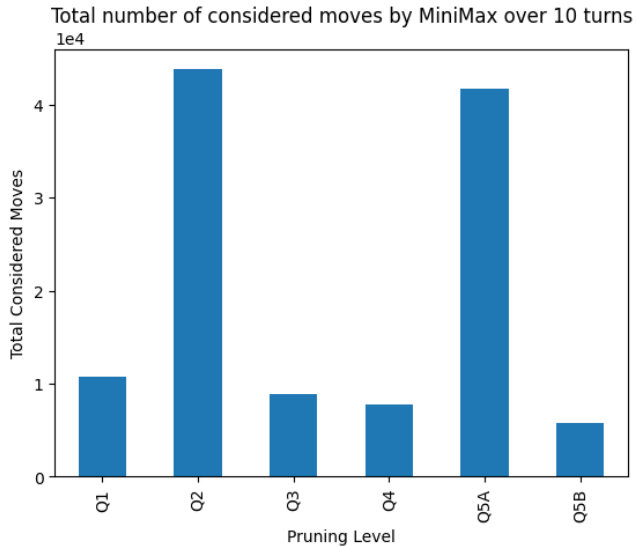
```
private static final int[] COLUMN_ORDER
= {3, 2, 4, 1, 5, 0, 6};
private Move[] reorder(Move[] moves) {
    Move[] ordered = new Move[moves.
        length];
    for (int i = 0; i < COLUMN_ORDER.
        length; i++)
        ordered[i] = moves[COLUMN_ORDER
            [i]];
    return ordered;
}
```

Results: All Variants at Depth 4

Move	Q1 original	Q2 pure minimax	Q3 $\geq$ prune	Q4 deep α - β	Q5a pure+order	Q5b deep+order
1	1426	4680	1208	993	4680	302
2	925	4680	771	642	4680	360
3	654	4680	574	514	4650	702
4	809	4623	745	612	4538	899
5	1221	4622	1158	934	4650	872
6	2459	4056	2007	1936	4649	773
7	909	4544	854	754	4333	785
8	760	4256	736	664	4325	671
9	899	3978	793	665	2792	305
10	664	3641	59	59	2347	95
SUM	10726	43760	8905	7773	41644	5764

Q5b (5764) examines $7.6\times$ fewer positions than Q2 (43760) and $1.9\times$ fewer than Q1 (10726). For Q1-Q4 the player moves were equal, at Q5a and Q5b the player's moves had to be adopted to not lose in < 10 moves.

Results: Visual Comparison



Q2 and Q5a (no pruning) dominate. Each pruning step reduces counts; Q5b achieves the lowest.

Q6: Reducing to 7 Columns

Change the constant

```
// C4Board.java  
public static final int  
    NUMBER_OF_COLUMNS = 7;
```

All array sizes and loop bounds are derived from this constant.

Update move-ordering array

```
// 7-column centre-out  
private static final int[]  
    COLUMN_ORDER =  
    {3, 2, 4, 1, 5, 0, 6};
```

Magic number 84

One literal not derived from constants:

```
rows = new Vector<C4Row>(84);
```

84 = four-slot groups on a 6×8 board.

Formula: $R(C-3) + (R-3)C + 2(R-3)(C-3)$

Direction	6×8	6×7
Horizontal	30	24
Vertical	24	21
Diagonals	30	24
Total	84	69

Q7: What the Heuristic Counts

Open groups

A group of 4 consecutive slots containing tokens of *only one* player.
(Mixed groups are dead — counted for neither player.)

Tracked separately: open groups of size 1, 2, 3.

```
// SHIFT_3_IN_A_ROW = 5  =>  x32  
// SHIFT_2_IN_A_ROW = 2  =>  x4  
return (p1_3InARow << 5)  
      + (p1_2InARow << 2)  
      + p1_1InARow;
```

Weights: 32 : 4 : 1

Special cases

- 4-in-a-row $\Rightarrow$ **Integer.MAX_VALUE** (win)
- Opponent has 4-in-a-row $\Rightarrow$ raw strength = 0

Final heuristic value

$\text{getStrength}(p) = \text{str}_p - \text{str}_{\text{opp}}$

Positive = good for player p ,
negative = bad.

Q7: Observer Pattern --- Incremental $O(1)$ Evaluation

C4Slot (Subject)

- Holds one cell's content
- Fires **contentsChanged()** to all listeners on every write

C4Row (Observer)

- Represents one group of 4 consecutive slots
- Registered on all 4 slots at construction
- Updates group counters, calls **Inc/Dec** on the shared **C4Stats** accumulator — board-wide counters updated in $O(1)$

Call chain on every move

```
board.move()  
→ slot.setContents(n)  
→ notifies each C4Row  
→ row.contentsChanged()  
→ stats.pX_NInARowInc()
```

undoLastMove() triggers the same chain in *reverse* — no board scan needed.

Why it matters

Each **board.move()** and **undoLastMove()** costs $O(1)$ for the heuristic — no full board scan at every minimax node.

Q8: Why the Computer Misses the Immediate Win

Root cause

Every win scores `Integer.MAX_VALUE` regardless of depth — a win in 1 move and a win in 3 moves look *identical*.

Move-selection loop

```
if (strength > maxStrength) {  
    maxStrength = strength;  
    maxIndex = i;  
}
```

Updates only on a *strict* improvement — ties are ignored.

Scenario (left-to-right ordering)

1. Col 0 tried first — search finds a forced win 3 moves deep. Returns `MAX_VALUE`. `maxIndex = 0`.
2. Col 3 is the immediate win. Also returns `MAX_VALUE`.
3. `MAX_VALUE > MAX_VALUE` is false — `maxIndex` stays at 0.
4. Computer plays col 0: wins, but not immediately.

Potential Fix

Encode depth in the score: `MAX_VALUE - depth`, so shallower wins always outrank deeper ones.

Summary

Pruning progression

Q2 (no pruning)	43 760	100%
Q1 (shallow α - β)	10 726	-75%
Q3 (equal cuts)	8 905	-79%
Q4 (deep α - β)	7 773	-82%
Q5b (+ ordering)	5 764	-87%

Move ordering (Q5b) gives the largest single reduction: good columns first $\Rightarrow$ tight window $\Rightarrow$ massive pruning.

Key takeaways

- Pure minimax visits $\frac{n^{d+1}-n}{n-1}$ nodes — exponential in depth
- Every pruning refinement compounds the savings
- Move ordering *without* pruning is irrelevant (Q5a $\approx$ Q2)
- A single constant controls board size; one magic number needed a formula
- Flat win scores cause occasional missed immediate wins